F R A M E F L O W O S

Plano Técnico de Integração Progressi va

PROMPT 05 — Integração da experiência visual aprovada (Conceito C) ao aplicativo FRAME FLOW, preservando integralmente a fundação técnica das Etapas 1 e 1.1

Projeto: FRAME FLOW — Sistema operacional da FRAME VISION

Escopo: somente o plano técnico das Fases 1, 2 e 3

Estado: nenhum arquivo do aplicativo foi modificado

Data: 30 de outubro de 2026 · Aguardando autorização

Sumário

Sumário Executivo..... 1

1. Estado Atual Verificado..... 2

1.1 Fundação técnica verificada em disco.....2

1.2 Correção de premissas históricas.....2

1.3 O protótipo aprovado está versionado no repositório.....3

1.4 Pendências reais e insumos externos.....3

2. Fase 1 — Planejamento de Integração.....3

2.1 Comparação módulo a módulo: protótipo x componentes existentes.....4

2.2 O que pode ser reutilizado sem alteração.....4

2.3 Estratégia de transformação: HTML/CSS/JS do protótipo em React/TypeScript.....4

2.4 Mapeamento dos indicadores demonstrativos para fontes reais no Supabase.....6

2.5 Separação: componentes puramente visuais x componentes com regras de negócio….6

2.6 Riscos de regressão documentados..... 7

2.7 Critérios de rollback..... 7

3. Fase 2 — Validação Técnica.....9

3.1 Conexão com o Supabase, sem credenciais nem dados fictícios.....9

3.2 Revisão das migrations endurecidas na Etapa 1.1.....10

3.3 Os sete testes de integração RLS pendentes.....10

3.4 Segurança de segredos no frontend.....11

4. Fase 3 — Prova de Conceito Integrada..... 11

4.1 Escopo estrito e matriz de preservação.....11

4.2 Dos dados demonstrativos aos estados reais.....12

4.3 Sequência de implementação..... 13

4.4 Gates de qualidade obrigatórios — como serão executados e apresentados.....13

4.5 Aceite e rollback da Fase 3.....13

5. Consolidado — Esforço, Sequência e Marcos.....14

5.1 Matriz de esforço consolidada.....15
5.2 Sequência com pontos de autorização.....15
5.3 PENDENTE DE VALIDAÇÃO — verificado agora x medível depois.....15
5.4 Próximos passos.....16

Nota: este Sumário é gerado por códigos de campo. Após qualquer edição, clique com o botão direito sobre o Sumário e escolha Atualizar Campo para atualizar os números de página.

Sumário Executivo

Este documento responde ao PROMPT 05 e entrega exclusivamente o **plano técnico de integração progressiva** da experiência visual aprovada — o Conceito C consolidado, avaliado no protótipo do PROMPT 04 — ao aplicativo FRAME FLOW existente. Nenhum arquivo do aplicativo principal foi modificado para produzir este plano: a árvore de fontes em `src/` e as migrations permanecem exatamente como estavam no último commit, com a árvore de trabalho limpa. Os únicos artefatos novos desta entrega são este documento e os scripts que o geram, ambos fora do código do aplicativo.

A verificação em disco corrigiu premissas importantes herdadas do histórico do projeto. As **oito correções técnicas da Etapa 1.1 já estão aplicadas** e auditáveis no repositório: o tipo de prioridade de tarefas voltou a ser exportado, a validação de tipos está ativa no build, a view financeira usa security_invoker, os votos de sócios vivem em tabela relacional com RLS por identidade, todas as funções têm search_path fixado e a suíte de testes automatizados está instalada, com 84 testes aprovados e 7 pulados. O que de fato permanece pendente é menor e bem delimitado: a suíte de integração RLS nunca foi executada contra um banco real — ela pula sozinha enquanto as variáveis de ambiente não existirem — e o arquivo `.env.local` não foi criado, por ser insumo exclusivo do contratante.

O plano segue as três fases exigidas. A **Fase 1** — comparação de componentes, inventário de reutilização, estratégia de transformação, mapeamento de indicadores para fontes reais, separação entre visual e regra de negócio, riscos de regressão e critérios de rollback — é entregue executada, por análise documentada com base em leitura direta dos arquivos. A **Fase 2** prepara a validação técnica: conexão Supabase sem criar credenciais ou dados fictícios, revisão formal das migrations endurecidas, especificação de execução dos sete testes de integração RLS e auditoria de segredos no frontend. A **Fase 3**, que depende de autorização específica, substituirá somente a tela inicial do CEO por composição protegida por flag, preservando integralmente login, controle de acesso, navegação entre ambientes, dados compartilhados, funcionalidades comerciais e operacionais, PWA e testes existentes.

O esforço estimado é de **6 a 8 horas para a Fase 2** e de **66 a 97 horas para a Fase 3** (76 a 112 horas com reserva de contingência de 15%), além de insumos que só o contratante pode fornecer: o projeto Supabase de teste com quatro usuários de teste, as variáveis de ambiente e, para os gates finais de desempenho, o ambiente de publicação. As estimativas assumem um único desenvolvedor sênior familiarizado com a base, com precisão declarada de mais ou menos 30%. Os pontos de autorização são explícitos: nada da Fase 2 será executado antes de autorização, e a Fase 3 exige autorização adicional e específica, nos termos do PROMPT 05.

1. Estado Atual Verificado

1.1 Fundação técnica verificada em disco

O inventário abaixo foi obtido por leitura direta dos arquivos do repositório — não por memória, nem por documentação. Ele estabelece a linha de base contra a qual qualquer integração será medida e explica por que este plano parte de evoluir a fundação existente, e não de recriar sistema algum. A fundação compreende oito módulos de biblioteca em `src/lib/frameflow/`, o cliente Supabase com fluxo PKCE, o shell responsivo com três ambientes, quarenta e sete primitivas de interface, a camada PWA e seis migrations SQL com políticas RLS completas.

Tabela 1 — Fundação do FRAME FLOW — verificação atual

Ativo	Estado verificado
`supabase/migrations/`	Seis migrations (0001 a 0006): perfis e permissões com trigger de criação de perfil e anti-elevação de cargo; comercial; operacional; financeiro com view blindada; decisões com votos relacionais e histórico protegido por trigger; engine de automações idempotente.
Camada de dados	DataProvider tipado em `data.tsx` com mappers de centavos para BRL, conversão de convenções de nomes e estados honestos (ListResult) para carregando, vazio, erro e não configurado.
Autenticação	Cliente Supabase com fluxo PKCE persistido (chave frameflow.auth), login por e-mail e senha, modo de pré-visualização restrito ao estado sem configuração e nunca gerador de sessão.
Shell e ambientes	Sidebar recolhível no desktop, bottom-nav com cinco atalhos por ambiente no mobile, troca de ambiente sem novo login e trinta e seis módulos mapeados com fase.
Tela inicial do CEO atual	Nove componentes em src/components/executivo/ (Direção Criativa 02): Command Header, Cap Intelligence, Financial Intelligence, Operational Pulse, Decision Center, paleta de comandos, véu de boot cinematográfico e hook de derivação de dados.
Testes automatizados	Vitest 3 com Testing Library e jsdom: oito suítes unitárias (84 testes aprovados) e uma suíte de integração RLS com sete testes que pulam sozinha sem banco de teste.
PWA	Manifest gerado por código, service worker que mantém Supabase e autenticação fora do cache, ícones gerados, botão de instalação e registro apenas em produção.

Ativo	Estado verificado
Qualidade da Etapa 1.1	ESLint sem erros, verificação de tipos sem erros, build de produção com validação de tipos ativa, smoke test HTTP e verificação em navegador nos formatos desktop e mobile, conforme documentado em docs/STATUS.md.

1.2 Correção de premissas históricas

O histórico do projeto registrou as oito correções da Etapa 1.1 como autorizadas e ainda não executadas. A verificação atual demonstra que elas **já foram executadas e validadas**. A tabela a seguir confronta cada afirmação anterior com o estado verificado e a evidência encontrada no código. Esse acerto de contas importa porque o plano não pode recomendar trabalho já feito, nem omitir trabalho realmente pendente.

Tabela 2 — Afirmações anteriores x estado verificado no repositório

Afirmação anterior	Estado	Evidência verificada
TaskPriority não exportado de types.ts	Corrigido	O tipo está exportado em src/lib/frameflow/types.ts e é usado pelo schema de tarefas.
ignoreBuildErrors ativo no next.config.ts	Corrigido	A configuração contém apenas output standalone e reactStrictMode falso; comentário local declara que o build valida tipos.
tsconfig.json incluindo scaffold no include	Corrigido	O include cobre src, tests e configurações; pastas de scaffold aparecem apenas no exclude.
View da migration 0004 sem security_invoker	Corrigido	vencimentos_em_aberto é criada com security_invoker ativo e grants revogados de anon e public.
Votos armazenados em coluna JSONB (0005)	Corrigido	Os votos vivem na tabela relacional decisao_votos, com chave primária dupla e RLS por identidade; o histórico permanece JSONB, porém blindado por trigger.
Função eh_interno sem search_path fixado	Corrigido	Todas as funções SQL declaram search_path explícito, conforme a regra de segurança do Supabase.
Nenhum framework de testes instalado	Corrigido	Vitest, Testing Library e jsdom estão nas dependências de desenvolvimento, com vitest.config.ts e nove suítes em tests/.
Arquivo .env.local inexistente	Confirmado	Existe apenas .env.example, listando somente os nomes das duas variáveis públicas exigidas. É insumo do contratante, não pendência de engenharia.

Duas pendências permanecem reais e inerentes ao processo: a suíte de integração RLS ainda não teve um banco Supabase real contra o qual rodar e o aplicativo ainda não foi conectado a projeto algum. Nenhuma delas bloqueia este planejamento; ambas têm lugar definido na Fase 2, com pré-requisitos e método especificados no capítulo 3. Enquanto as variáveis de ambiente não forem fornecidas, a suíte de integração continua pulando de forma automática e honesta, e o aplicativo continua exibindo seus estados não configurados com dignidade, sem qualquer número fictício.

1.3 O protótipo aprovado está versionado no repositório

O protótipo do PROMPT 04 está presente em dois locais: em `download/frameflow-prototipo-prompt04/`, como pacote de entrega com o relatório em PDF e treze capturas de tela, e em cópia idêntica em `public/prototypes/`, dentro da pasta servida pelos builds de produção. Os ativos técnicos do protótipo são sete arquivos: `common.css` com os tokens compartilhados, as folhas desktop e mobile, os roteiros de interação desktop e mobile, o dataset demonstrativo rotulado em `data.js` e o motor do núcleo orbital em `orbital.js`. A composição aprovada compreende quatro contextos (Visão Geral, Inteligência Financeira, Central de Decisões e Ecossistema), dois modos (Comando e Imersivo), a paleta de comandos com doze ações em seis grupos, o dock da Cap com quatro estados demonstrativos e o núcleo orbital desenhado em canvas 2D com projeção 3D própria, sem dependências externas.

Um ponto de higiene de implantação é levantado como risco R13 no capítulo 2: a cópia em `public/prototypes/` adiciona 4,2 megabytes a cada bundle de produção, porque o processo de build copia a pasta pública para o artefato autônomo. O plano recomenda removê-la da pasta pública durante a Fase 3, mantendo o pacote íntegro em download/ e no histórico do git. A decisão, contudo, pertence ao contratante, pois a cópia pública pode estar servindo de referência a sócios e avaliadores. Se mantida, o custo é apenas de peso, sem risco funcional.

1.4 Pendências reais e insumos externos

Encerrado o acerto de premissas, as pendências reais do projeto são exclusivamente operacionais e dependem de insumos do contratante. A tabela abaixo registra cada uma, o responsável natural e o momento do plano em que se resolve. Nenhuma delas é criada ou simulada por esta engenharia: o PROMPT 05 é explícito ao proibir credenciais, contas ou dados fictícios, e este plano cumpre a proibição ponta a ponta.

Tabela 3 — Pendências reais — responsável e momento de resolução

Pendência	Responsável	Quando resolve
Projeto Supabase real e variáveis em .env.local (duas variáveis públicas)	Contratante	Antes ou durante a Fase 2; guia existente em docs/SETUP-SUPABASE.md
Projeto Supabase de teste descartável com as seis migrations e quatro usuários de teste	Contratante	Fase 2 — pré-requisito da suíte RLS
Execução da suíte de integração RLS (sete testes) com evidências	Engenharia	Fase 2, imediatamente após os insumos acima
Assets oficiais da Cap (arte 2D aprovada)	Contratante	Fase 3 — a área integrada permanece com arte de referência rotulada
Ambiente de publicação para os gates de desempenho	Contratante	Fase 3, etapa final — sem ele, Lighthouse é medido em build local com ressalva

Nota de escopo: este documento não altera o aplicativo, não executa migrations, não cria contas e não gera dados. Toda execução começa somente após as autorizações previstas nos marcos do capítulo 5.

2. Fase 1 — Planejamento de Integração

A Fase 1 foi executada como análise documentada, por leitura cruzada dos sete ativos do protótipo com os componentes React do aplicativo, sem qualquer modificação no código. O método foi: enumerar cada módulo do protótipo a partir dos roteiros de interação e das folhas de estilo; localizar o correspondente funcional no aplicativo; classificar o veredicto entre estender, evoluir, reutilizar ou criar; e definir a estratégia de tradução de cada um. O resultado é o comparativo, o inventário de reutilização e a estratégia de transformação das seções a seguir, seguidos do mapeamento de indicadores, da separação de responsabilidades, dos riscos de regressão e dos critérios de rollback.

2.1 Comparação módulo a módulo: protótipo x componentes existentes

A tela inicial do CEO atual já segue a Direção Criativa 02, com cinco regiões cinematográficas em grafite e dados reais com estados honestos. A integração, portanto, não parte do zero: é a evolução dessa composição para a linguagem Signature aprovada no PROMPT 04, acrescentando os módulos que só existem no protótipo — contextos, modos e núcleo orbital — e revestindo os painéis existentes com a caligrafia editorial do Conceito C consolidado, que funde a estrutura funcional de C com a elegância editorial de A e o núcleo orbital de B como recurso opcional.

Tabela 4 — Módulos do protótipo x correspondentes no aplicativo — veredicto e estratégia

Módulo do protótipo	Correspondente atual no app	Veredicto	Estratégia
Tokens de common.css (escada de contraste, painéis por modo, serif e mono)	src/app/globals.css — tokens ink, paper e vermelho, utilitários ff, já em Tailwind v4	Estender	Adição exclusivamente aditiva: escada de opacidades, fundos de painel por modo, utilitários serifados e monoespaçados; nada é removido; paleta e easing idênticos aos da marca.
Famílias Instrument Serif e JetBrains Mono (via CDN no protótipo)	Apenas Inter via next/font no layout	Novo no app	Carregar as duas famílias com next/font auto-hospedado; o import de CDN do protótipo não é portado porque o aplicativo é um PWA que deve funcionar offline.
Quatro contextos com navegação própria por hash (painel, financeiro, decisões, ecossistema)	Tela única com cinco regiões em src/components/views/executivo/inicio.tsx	Novo no app	Contextos como estado interno do componente, navegáveis por trilho visual e pela paleta; o hash do protótipo não é portado (ver 2.3, decisão 3).
Modos Comando e Imersivo com body por atributo de dados	Não existe; apenas o véu de boot pontual por sessão	Novo no app	Provedor de modo com persistência por flag de armazenamento; padrão Comando no servidor; Imersivo ativa a camada 3D (ver 2.3, decisão 4).
Núcleo orbital (orbital.js, canvas 2D com projeção 3D)	Não existe	Novo no app	Porte do motor para módulo TypeScript isolado e componente React; nós e arestas derivados apenas de relações reais; todas as regras de movimento aprovadas preservadas.
Painel financeiro editorial (KPIs, série de 6 meses, categorias, contas, alertas, aportes)	src/components/executivo/financial-intelligence.tsx — quatro KPIs, série mensal e slots honestos	Evoluir	Revestimento editorial mais derivações novas (caixa, runway, categorias, contas com nome de cliente); aportes e metas permanecem slots honestos por não terem fonte real.

Módulo do protótipo	Correspondente atual no app	Veredicto	Estratégia
Central de decisões com prévia, filtros, gaveta de detalhe e votação	src/components/executivo/decision-center.tsx e views/executivo/decisoes.tsx com voto real	Evoluir	Nova composição visual reutilizando a votação real via castVote; gaveta no padrão de interface do app; pedir contexto vira abstenção com comentário, que já existe no modelo.
Pipeline comercial e estatísticas na visão geral do CEO	Existe no ambiente Comercial (comercial/inicio.tsx)	Reutilizar dado	Composição nova na tela do CEO lendo leads pelo DataProvider; nenhuma duplicação de lógica, apenas nova derivação no hook do CEO.
Pulso operacional com projetos, contadores e linha da semana	src/components/executivo/operational-pulse.tsx	Evoluir	Revestimento editorial e acréscimo da linha da semana derivada de prazos de tarefas e projetos.
Dock e painel da Cap com quatro estados demonstrativos	Máquina real de oito estados por quatro apresentações em cap.tsx e cap-panel.tsx	Reutilizar máquina	Mapear os quatro rótulos demonstrativos para estados reais da máquina; o componente da Cap permanece intocado; a área oficial segue rotulada.
Paleta de comandos com doze ações em seis grupos	src/components/executivo/command-palette.tsx sobre cmdk	Estender	Acrescentar comandos de contexto, de alternância de modo e de acessibilidade, mantendo módulos, ambientes e ações rápidas existentes.
Camada cinematográfica (grão, vinheta, letterbox, cantoneiras, perfuração)	Véu de boot e utilitários ff existentes	Novo no app	Camada decorativa isolada, ativa apenas no modo Imersivo, congelada em movimento reduzido, sem peso perceptível de bundle.
Toasts e sobreposições com fechamento por Esc empilhado	Sonner instalado e diálogos Radix existentes	Reutilizar	Revestimento visual do sonner já instalado; a semântica de Esc fechando o topo da pilha é reproduzida nos manipuladores React.

2.2 O que pode ser reutilizado sem alteração

A fundação é o ativo a preservar. Todo o núcleo de domínio, autenticação, roteação, formatação, armazenamento, shell e componentes de interface permanecem intocados pela

integração — não por conveniência, mas porque concentram as garantias testadas do sistema: estados honestos, centavos convertidos nas bordas, troca de ambiente sem novo login e políticas RLS prontas para validação. A tabela abaixo formaliza o inventário de reutilização direta, que serve também de lista de proibições de toque para a Fase 3.

Tabela 5 — Inventário de reutilização direta (sem alteração)

Grupo	Arquivos	Motivo da preservação
Núcleo de domínio	`src/lib/frameflow/{types,environments,router,format,data,auth,storage,schemas}`	Tipos, ambientes, roteador por hash, formatação BRL, camada de dados honesta, sessão PKCE e esquemas Zod concentram as regras testadas do sistema.
Cliente Supabase	`src/lib/supabase/{client,config}`	Validação estrita de variáveis, fluxo PKCE e chave de armazenamento de sessão; qualquer toque aqui é risco de autenticação.
Shell responsivo	`src/components/shell/` (app-shell, topbar, sidebar, bottom-nav, preview-banner)	Navegação entre ambientes sem novo login e adaptação mobile já verificadas em 390 e 1440 pixels.
Biblioteca de interface	47 componentes em `src/components/ui/`	Base shadcn de onde saem diálogos, gavetas, formulários e toasts usados pela nova composição.
PWA	`src/app/manifest.tsx`, `public/sw.js`, `src/components/pwa/`	Instalação e cache já corretos; a Fase 3 apenas eleva a versão de cache no release, preservando as estratégias.
Cap	`src/components/cap/{cap,cap-panel}`	Máquina de oito estados por quatro apresentações e slot de assets oficiais; nenhum novo mascote, nenhuma arte inventada.
Views de negócio	`src/components/views/(comercial, operacional, decisoes, indicadores, financeiro, login, configuracoes)`	Funcionalidades comerciais e operacionais do MVP permanecem intactas; a prova de conceito não as toca.
Testes existentes	`tests/unit (8 suítes) e tests/integration`	84 testes aprovados e a suíte RLS são a rede de segurança da integração; precisam continuar verdes sem edição.

Grupo	Arquivos	Motivo da preservação
Migrations	`supabase/migrations/0001 a 0006`	Esquema endurecido na Etapa 1.1; a prova de conceito não exige nenhuma mudança de banco.

2.3 Estratégia de transformação: HTML/CSS/JS do protótipo em React/TypeScript

A tradução segue sete decisões explícitas, cada uma tomada para eliminar uma classe de risco antes que ela exista. Elas valem como especificação vinculante para a Fase 3: qualquer implementação que as contrarie deve ser tratada como desvio de escopo.

1. **Tokens primeiro, por extensão aditiva.** A escada de contraste, os fundos de painel por modo e os utilitários tipográficos de `common.css` entram em globals.css sem remover linha alguma; a paleta e o easing já são os da marca, o que reduz a superfície de mudança ao mínimo necessário.
2. **Fontes auto-hospedadas.** Instrument Serif e JetBrains Mono entram via next/font com subconjuntos, no lugar do import de CDN do protótipo; o aplicativo é PWA e precisa renderizar offline; o peso adicional é medido no gate de desempenho.
3. **Contextos como estado interno.** O roteador do aplicativo usa hash no formato ambiente seguido de módulo, com deep links já em uso; portar o hash de contexto do protótipo quebraria o roteador. Os quatro contextos vivem como estado do componente, navegáveis pelo trilho e pela paleta; extensão futura do roteador para contexto profundo fica registrada como opcional e fora da prova de conceito.
4. **Modos com padrão seguro.** O servidor renderiza sempre o modo Comando; a escolha do usuário persiste por flag de armazenamento já existente, com chave nova; quem pede movimento reduzido ao sistema operacional permanece em Comando. Assim não há falha de hidratação nem custo de render 3D no carregamento inicial.
5. **Orbital isolado e fiel.** O motor de canvas com projeção 3D própria é portado para um módulo TypeScript sem dependências; preservadas as regras aprovadas: câmera por presets discretos com interpolação, pausa total em modo Comando e aba oculta, cena estática em movimento reduzido, degradação adaptativa de resolução e densidade.
6. **Interações no paradigma React.** O estado vira React; os toasts migram para o sonner já instalado com revestimento editorial; os diálogos usam as primitivas Radix existentes; a gaveta de decisão usa sheet no desktop e drawer no mobile, com foco preso; Esc continua fechando o topo da pilha de sobreposições.

7. **Tipagem estrita e zero dependências novas.** Todos os componentes tipam contra os tipos de domínio existentes, sem any e sem supressões; nenhuma dependência nova entra no package.json — motor orbital, cmdk e sonner já cobrem tudo o que o protótipo demonstra.

2.4 Mapeamento dos indicadores demonstrativos para fontes reais no Supabase

O dataset demonstrativo do protótipo (`data.js`, rotulado como cem por cento fictício) não é portado ao aplicativo. Cada indicador que ele exibe recebe uma fonte real no Supabase, uma derivação nova sobre dados já lidos ou, quando não existe fonte, um slot honesto rotulado — porque o aplicativo jamais apresenta número inventado como resultado da empresa. O mapeamento completo segue abaixo; ele é a base do contrato de dados da etapa S2 na Fase 3.

Tabela 6 — Indicador demonstrativo x fonte real no Supabase x situação

Indicador do protótipo	Fonte real	Situação
Receita do mês	transacoes_financeiras com tipo entrada e vencimento no mês corrente, soma de valor em centavos	Já existe: receitaMes no hook do CEO
Despesas do mês	transacoes_financeiras com tipo saída no mês corrente	Já existe: despesasMes
Resultado operacional	receita do mês menos despesas do mês	Já existe: resultadoOperacional
Contas a receber e contas vencidas	entradas não pagas; vencidas por comparação de data	Já existe: contasReceber e vencidas
Serie de 6 meses	agrupamento de transações por mês de vencimento	Já existe: serieMensal
Caixa	soma de entradas pagas menos saídas pagas	Derivação nova em use-dados-ceo, sem nova tabela
Runway	caixa dividido pela média de saídas dos últimos seis meses	Derivação nova; nulo quando o histórico for insuficiente, com estado honesto
Despesas por categoria	agrupamento por finance_category (doze categorias do enum)	Derivação nova
Contas por cliente com nota fiscal	transações não pagas com junção de projeto e cliente para exibir nome	Extensão aditiva de leitura: junção de nome de cliente

Indicador do protótipo	Fonte real	Situação
Alertas financeiros	contas vencidas e decisões com prazo próximo	Derivação nova, reutilizando os sinais que a Cap já calcula hoje
Aportes de capital	não há tabela para aportes na fundação	Slot honesto rotulado de Fase 2 — sem número fictício
Meta trimestral	não há tabela de metas na fundação	Slot honesto rotulado de Fase 2
Pipeline comercial por etapa	leads agrupados por etapa com valor estimado em centavos	Dado já existente; composição nova na tela do CEO
Estatísticas comerciais	derivações sobre leads e clientes	Derivação nova no hook do CEO
Projetos ativos e principais projetos	projetos por status, excluindo entregues e pausados	Já existe: operacional.projetosAtivos e topProjetos
Entregas em 14 dias, aprovações, tarefas atrasadas	projetos e tarefas por datas e status	Já existe no hook do CEO
Linha da semana	prazos de tarefas e projetos distribuídos nos sete dias	Derivação nova
Decisões abertas com regra, prazo, impacto e votos	decisoes com junção de decisao_votos já implementada em listDecisions	Já existe; voto real via castVote
Nós do ecossistema	projetos ativos, decisões abertas, indicadores-chave e alertas derivados dos dados acima	Derivação nova; orbital desenha apenas relações reais
Arestas do ecossistema	chaves estrangeiras reais: projeto-cliente, transação-projeto, decisão-núcleo	Derivação nova a partir das mesmas leituras
Comandos da paleta	ações reais existentes: navegação, votação, Cap, modos, acessibilidade	Nova definição estática ligando a ações reais
Estados da Cap (quatro rótulos demonstrativos)	máquina real de oito estados: atenção para alerta, processando para espera, alerta para alerta com outra fala, confirmação para celebração	Mapeamento de rótulos; máquina intacta

Três consequências práticas desse mapeamento orientam a Fase 3: o hook `use-dados-ceo.ts` ganha derivações novas sem perder o contrato de pronto e nulos honestos; a leitura de transações ganha uma junção aditiva opcional para exibir nome de cliente, sem alterar os mappers testados; e dois indicadores do protótipo — aportes e metas — permanecem slots

honestos até existir modelo de dados, porque inventar fonte seria violar a regra de honestidade do produto.

2.5 Separação: componentes puramente visuais x componentes com regras de negócio

A fronteira é mecânica e verificável em revisão: componente visual não conhece Supabase. Tudo o que ele exibe chega por propriedades; tudo o que ele dispara chega por retornos de chamada. As regras de negócio — derivações, votação, acesso e roteação — vivem nos hooks e módulos já existentes, que a integração apenas estende. Essa separação permite revestir a interface sem risco funcional e testar cada lado por métodos próprios: snapshot e acessibilidade para o visual; unidade e integração para as regras.

Tabela 7 — Classificação visual x regra de negócio

Componente	Classificação	Contrato
Painéis editoriais, KPIs, hairlines, cantoneiras, grão, letterbox	Visual puro	Recebem valores prontos; nenhum acesso a dados; nenhuma decisão de negócio.
Núcleo orbital e motor de canvas	Visual com entrada	Recebe nós e arestas prontos por propriedades; ignora a origem dos dados; só desenha relações recebidas.
Hook do CEO estendido	Regra de negócio	Única fonte de derivação dos indicadores; mantém o contrato de pronto e nulos; sem acesso a componentes.
Central de decisões com gaveta	Regra com revestimento	Votação via castVote com identidade do sócio autenticado; voto em nome de outro é bloqueado pelo banco.
Paleta de comandos estendida	Regra	Navegação pelo roteador existente; modos pelo provedor de modo; nenhuma navegação improvisada.
Provedor de modo	Regra leve	Persistência por flag de armazenamento; movimento reduzido do sistema vence; nenhum dado sensível.

A estrutura de pastas proposta respeita a mesma fronteira: a camada visual nova vive em `src/components/executivo/signature/`; as derivações estendem `src/components/executivo/use-dados-ceo.ts`; e os testes novos entram em `tests/unit/` espelhando as suítes existentes. Nenhum arquivo da fronteira de regras é movido ou renomeado — apenas estendido de forma aditiva, o que mantém o histórico de revisões legível e o risco de conflito baixo.

2.6 Riscos de regressão documentados

Cada risco abaixo tem causa identificada, mitigação planejada e método objetivo de detecção — porque risco sem detecção é só esperança. Probabilidade e impacto foram classificados à luz do estado verificado da fundação; a lista é revisada ao final de cada etapa da Fase 3, e qualquer risco que se materialize entra no critério de rollback da seção 2.7.

Tabela 8 — Matriz de riscos de regressão

Id	Risco	Prob.	Impacto	Mitigação	Detecção
R1	Conflito entre o hash de contexto do protótipo e o roteador real do app	Baixa	Alto	Contextos como estado interno; roteador intocado (decisão 3 da seção 2.3)	Suítes de navegação existentes mais caminhada manual de deep links
R2	Service worker servindo shell antigo após a atualização	Média	Médio	Elevação da versão de cache no release da Fase 3; estratégia network-first para navegação já existente	Teste de atualização pós-release com verificação da nova versão ativa
R3	Falha de hidratação entre modo persistido e render do servidor	Média	Médio	Padrão Comando no servidor; escolha aplicada apenas após a montagem	Fumaça em navegador e teste de unidade do provedor de modo
R4	Peso de bundle (duas famílias de fonte e camada 3D) contra o alvo de desempenho	Média	Alto	next/font com subconjuntos; motor sem dependências; camada 3D carregada apenas sob demanda	Delta de bundle medido a cada etapa e gate de Lighthouse
R5	Contraste insuficiente nos novos tokens editoriais	Baixa	Alto	Escada de contraste do protótipo, com piso elevado aprovado no PROMPT 04	Medição por script de contraste reutilizado dos QA do protótipo
R6	Regressão nos 84 testes existentes	Baixa	Alto	Suítes intocadas; um PR atômico por etapa; proibições de toque da seção 2.2	Suíte completa executada a cada etapa antes do avanço
R7	Quebra de login ou do fluxo PKCE	Baixa	Crítico	Arquivos de autenticação intocados; matriz de preservação da seção 4.1	Fumaça de login, logout e sessão persistida

Id	Risco	Prob.	Impacto	Mitigação	Detecção
R8	Nova leitura com junção que contorne o RLS	Baixa	Crítico	Toda leitura pelo cliente anônimo sob RLS; a suíte T1 a T7 cobre a fronteira; service_role proibido	Reexecução da suíte RLS e revisão de código na fronteira de dados
R9	Erro de conversão centavos x BRL nos painéis novos	Baixa	Médio	Reuso obrigatório dos mappers e formatadores testados	Suíte de mappers existente mais testes das derivações novas
R10	Overflow horizontal em 390 pixels na nova composição	Média	Alto	Composição mobile-first herdada do protótipo mobile; trilho de contexto adaptado	Medição objetiva em 390, 768 e 1440 pixels na etapa final
R11	Quebra de deep links entre ambientes	Baixa	Alto	Roteador intocado; nenhum formato novo de URL	Abertura direta por URL em cada ambiente
R12	Custo de processamento do orbital em celulares modestos	Média	Médio	Pausa total fora do Imersivo e em aba oculta; degradação adaptativa; cena estática em movimento reduzido	Medidor interno de quadros por segundo e teste de pausa por visibilidade
R13	Protótipo servido em produção pesando 4,2 megabytes	Alta	Baixo	Remoção recomendada de public/prototypes na Fase 3, mantendo o pacote em download/ e no git; decisão do contratante	Tamanho do bundle de produção medido antes e depois

2.7 Critérios de rollback

O rollback é projeto, não improviso. A composição nova vive atrás de flag de recurso em tempo de build, com padrão desligado, e a composição atual permanece no código até a aceitação formal — nada é apagado. Com isso, voltar ao estado anterior tem dois caminhos: desligar a flag e regenerar, ou reverter o PR atômico da etapa incriminada; ambos ficam abaixo de trinta minutos em ambiente de build. Como a prova de conceito não altera esquema de banco, nenhum rollback toca dados; o repositório e o banco permanecem histórias independentes.

Os gatilhos abaixo são objetivos e disparam o rollback da etapa automaticamente, sem julgamento no meio do caminho:

1. Falha em qualquer gate obrigatório: ESLint, verificação de tipos, build ou suíte de testes.
2. Contraste abaixo de 4,5 para 1 em qualquer par obrigatório de texto da nova interface.
3. Overflow horizontal em qualquer largura verificada, notadamente 390 pixels.
4. Lighthouse mobile abaixo de 85 de performance ou de 90 de acessibilidade.
5. Qual quebra de login, logout ou do fluxo PKCE.
6. Quebra de instalação ou de atualização do PWA.
7. Falha em qualquer um dos sete testes de integração RLS.
8. Regressão de navegação por hash em qualquer ambiente.
9. Número fictício exibido como real — violação da regra de honestidade do produto.

Objetivo de restauração: flag desligada em minutos; reversão completa do PR em menos de trinta minutos. O banco de dados não participa do rollback da Fase 3 por não ser alterado por ela.

3. Fase 2 — Validação Técnica

3.1 Conexão com o Supabase, sem credenciais nem dados fictícios

O princípio é taxativo: nada é criado por esta engenharia — nenhum projeto, nenhuma conta, nenhuma chave, nenhum dado, fictício ou não. A conexão usa o contrato já implementado e testado: as duas variáveis públicas lidas por `src/lib/supabase/config.ts`, com validação estrita de URL em https, host supabase e formato de JWT, fornecidas pelo contratante em `.env.local` conforme o guia existente em `docs/SETUP-SUPABASE.md`. Enquanto essas variáveis não existirem, o aplicativo permanece em seu estado não configurado, honesto e rotulado, e o modo de pré-visualização segue disponível apenas nesse estado — comportamento já coberto por testes de unidade.

Para a suíte de integração, o banco é um projeto de teste descartável, criado e mantido pelo contratante. O runbook abaixo será materializado como `docs/VALIDACAO-RLS.md` na primeira hora da Fase 2 autorizada, e cada passo é do contratante — a engenharia somente prepara o roteiro, executa a suíte e registra as evidências:

- Criar um projeto Supabase de **teste descartável**, separado de qualquer dado real.
- Executar as seis migrations em ordem (0001 a 0006) no editor SQL do projeto de teste.
- Criar os **quatro usuários de teste** na autenticação com senhas fortes: sócio A (cargo ceo), sócio B (cargo cro), colaborador e cliente externo.

- Atribuir os cargos por SQL, usando o bloco SEED documentado na migration 0001 — o trigger anti-elevação bloqueia promoção pelo próprio cliente, por projeto.
- Definir as variáveis de teste: URL e chave anônima do projeto de teste mais as credenciais dos quatro usuários, conforme o cabeçalho da suíte.
- Executar a suíte completa e registrar o log integral como evidência.

Os quatro estados reais exigidos pelo PROMPT 05 — carregando, vazio, erro e conectado — já têm contrato implementado na fundação, via `ListResult`, `useAsyncList` e `EmptyState`, com o motivo de não configurado como cidadão de primeira classe. A Fase 2 formaliza esse contrato; a Fase 3 apenas o aplica à nova composição visual. Nenhum estado novo é inventado e nenhum dado de demonstração é promovido a dado real.

3.2 Revisão das migrations endurecidas na Etapa 1.1

A revisão por arquivo, abaixo, confirma que as seis migrations estão na forma endurecida descrita na Etapa 1.1 — a verificação foi feita por leitura direta do SQL, nesta entrega, sem executar nada contra banco. A conclusão é direta: **nenhuma mudança de esquema é necessária** para a prova de conceito da Fase 3, e a execução real dessas migrations no projeto de teste é o pré-requisito imediato dos sete testes de integração.

Tabela 9 — Migrations — conteúdo essencial e verificação

Arquivo	Conteúdo essencial verificado	Estado
`0001_perfis_e_permissoes.sql`	Enums de papel e ambiente; tabela de perfis ligada a auth.users; função eh_interno com search_path fixo; trigger de criação de perfil no primeiro login; trigger anti-elevação de cargo; RLS de leitura própria ou interna; bloco SEED de sócios documentado em comentário.	Endurecida
`0002_comercial.sql`	Clientes e leads com valor estimado em centavos; RLS interno para leitura e escrita via eh_interno.	Endurecida
`0003_operacional.sql`	Projetos e tarefas com chaves estrangeiras e orçamento em centavos; RLS interno para CRUD.	Endurecida
`0004_financeiro.sql`	Transações financeiras com valor em centavos e restrição de não negativo; view vencimentos_em_aberto com security_invoker ativo e grants revogados de anon e public; RLS interno.	Endurecida
`0005_decisoes.sql`	Decisões com impacto financeiro em centavos e histórico blindado por trigger; tabela relacional decisao_votos com chave primária dupla; RLS por identidade: sócio grava apenas o próprio voto, apenas em decisão aberta, sem política de exclusão.	Endurecida

Arquivo	Conteúdo essencial verificado	Estado
`0006_engine.sql`	Automações com unicidade por nome, entidade e identificador; trigger idempotente de venda fechada; RLS de leitura interna.	Endurecida

3.3 Os sete testes de integração RLS pendentes

A suíte existe e está pronta em `tests/integration/rls-supabase.test.ts`: usa a mesma biblioteca do aplicativo, autentica quatro clientes com sessões distintas e limpa os registros que cria. Sem as variáveis de ambiente, ela pula sozinha — comportamento verificado na última execução documentada, com sete testes reportados como pulados. Os sete testes, suas garantias e resultados esperados:

Tabela 10 — Sete testes de integração RLS — especificação e resultado esperado

Id	Garantia verificada	Resultado esperado
T1	O trigger de criação de perfil gerou perfil com cargo correto para o sócio autenticado	Perfil único do usuário com cargo ceo, sem erro
T2	Anti-elevação: usuário autenticado não promove o próprio cargo	Atualização bloqueada com a mensagem de elevação de cargo
T3	Decisões: sócio cria; colaborador interno lê e não cria; cliente externo não lê nada	Criação do sócio sem erro; leitura do colaborador; bloqueio da criação do colaborador; zero linhas para o externo
T4	Votos relacionais: cada sócio grava apenas o próprio voto; voto em nome de outro é bloqueado; alteração do voto alheio não altera nada	Inserção própria sem erro; inserção fraudulenta com erro; atualização alheia sem erro porém zero linhas afetadas
T5	Histórico de decisão imune a reescrita manual; mudança legítima de status segue funcionando	Reescrita manual falha com erro de histórico; mudança de status grava evento pelo trigger
T6	Financeiro: sócio lê tabela e view; cliente externo lê zero nas duas; colaborador interno lê	Uma linha para interno na tabela e na view; zero para o externo nas duas
T7	Engine idempotente: um único evento de venda fechada por lead, mesmo reentrando na etapa	Contagem exatamente igual a um em todos os cenários de reentrada

Método de execução: rodar a suíte completa com as variáveis de teste presentes e anexar o log integral, a data e a identificação do banco descartável como evidência. O resultado esperado é sete testes aprovados e nenhum pulado naquela execução. A higiene é herdada da própria suíte: leads e decisões criadas são removidas ao final de cada caso; eventos de automação são registrados por design sem política de exclusão, o que reforça a

recomendação de banco descartável. Se qualquer teste falhar, a falha é tratada como bloqueador da Fase 3 e diagnosticada antes de qualquer avanço — nunca relativa.

3.4 Segurança de segredos no frontend

A arquitetura de segurança é explícita: a chave anônima do Supabase é pública por design — trafega no frontend de qualquer aplicação Supabase — e a fronteira real de proteção é o conjunto de políticas RLS no banco, que é exatamente o que a suíte T1 a T7 valida. O que é proibido, sem exceção: a chave de função de serviço (service_role) em qualquer arquivo de `src/`, em qualquer variável com prefixo público, no service worker ou no bundle. O arquivo `.env.example` existe apenas com os nomes das duas variáveis públicas e uma advertência comentada sobre a chave de serviço; o `.gitignore` cobre arquivos de ambiente; o service worker mantém Supabase e autenticação fora do cache por estratégia de rede somente — tudo verificado nesta entrega.

Na Fase 2, a auditoria vira rotina executável: um roteiro de varredura estática confirma zero ocorrências de service_role em código e em pasta pública, confirma a cobertura do gitignore e confirma que nenhuma variável nova de ambiente entra sem nome documentado. O risco R13 conecta-se aqui: o build copia a pasta pública para o artefato autônomo, portanto nenhum segredo pode jamais residir ali. O fluxo PKCE persistido com a chave de sessão existente permanece intocado pela integração, e qualquer mudança futura nas estratégias de cache do service worker exige justificativa explícita em revisão.

4. Fase 3 — Prova de Conceito Integrada

4.1 Escopo estrito e matriz de preservação

A prova de conceito substitui exclusivamente a tela inicial do Ambiente Executivo, atrás de flag com padrão desligado. Tudo o mais é preservado por matriz verificável — cada item exigido pelo PROMPT 05 mapeado ao ativo concreto que o cumpre e ao método que comprovará a preservação no encerramento da fase. A área da Cap permanece com arte de referência rotulada, aguardando os assets oficiais; nenhum mascote é inventado.

Tabela 11 — Matriz de preservação — exigência, ativo e verificação

Exigência do PROMPT 05	Ativo concreto preservado	Verificação na Fase 3
Login e autenticação	Sessão PKCE em `auth.tsx`, cliente em `client.ts`, tela de login e gate por status	Fumaça de login, logout e sessão persistida; arquivos intocados na revisão

Exigência do PROMPT 05	Ativo concreto preservado	Verificação na Fase 3
Controle de acesso	Definições de ambiente em `environments.ts` e políticas RLS do banco	Reexecução da suíte RLS; nenhum papel novo criado
Navegação entre ambientes	`router.ts`, seletor de ambiente do topbar, troca sem novo login	Caminhada pelos três ambientes por clique e por URL direta
Dados compartilhados	DataProvider com mappers e estados honestos	Extensão apenas aditiva; suíte de mappers existente continua verde
Funcionalidades comerciais e operacionais	Views de comercial, operacional, decisões, indicadores e financeiro	Zero alteração nesses arquivos; listados como proibidos de toque
PWA	Manifest, service worker, registro apenas em produção e botão de instalação	Teste de instalação e de atualização com a versão de cache elevada
Testes existentes	Oito suítes unitárias e a suíte de integração RLS	Execução integral a cada etapa; qualquer vermelho é bloqueador

4.2 Dos dados demonstrativos aos estados reais

O dataset do protótipo não entra no aplicativo — nem valores, nem textos de contexto, nem votações simuladas. Cada região da nova tela consome as derivações reais do hook estendido e apresenta os quatro estados contratuais, já usados pela fundação e agora aplicados à linguagem editorial:

- **Carregando:** esqueleto editorial em silhueta, sem número provisório.
- **Vazio:** estado vazio com convite para a ação, no padrão do EmptyState existente.
- **Erro:** mensagem objetiva com botão de repetir, sem esconder a falha.
- **Conectado:** dados reais lidos pelo cliente sob RLS, convertidos dos centavos pela camada testada.

Indicadores sem fonte real — aportes e metas — permanecem slots honestos rotulados, visíveis como capacidades futuras, jamais como números. A regra de honestidade do produto vale para a camada visual nova exatamente como vale hoje: sem Supabase configurado, a tela é digna e clara sobre o que falta, com chamada para a configuração. Esse comportamento é o que separa a prova de conceito de uma maquete e é condição de aceite inegociável.

4.3 Sequência de implementação

Onze etapas, cada uma com entregas verificáveis, dependências explícitas, esforço estimado e um PR atômico. Nenhuma etapa avança com a anterior vermelha em qualquer gate; a flag permanece desligada até a etapa final, e as camadas visuais novas convivem com a composição atual no mesmo pacote, sem custo de render para o usuário enquanto desligadas.

Tabela 12 — Sequência S1 a S11 — entregas, dependências e esforço

Etapa	Entregas e arquivos afetados	Depende de	Esforço
S1	Tokens e fontes: extensão aditiva de `globals.css` (escada de contraste, painéis por modo, utilitários serif e mono, cantoneiras, perfuração, grão, vinheta, letterbox) e `layout.tsx` com next/font para as duas famílias novas	Nenhuma	4 a 6 h
S2	Camada de dados: derivações novas em `use-dados-ceo.ts` (caixa, runway, categorias, semana, pipeline, contas com nome de cliente); junção aditiva opcional de leitura em `data.tsx`; testes unitários das derivações	S1	6 a 9 h
S3	Primitivas editoriais em `src/components/executivo/signature/`: painel editorial, KPI, barra de segmentos de voto, chips	S1	4 a 6 h
S4	Contextos e integração ao shell: trilho de contexto, cabeçalho editorial fundido ao Command Header existente, ramo por flag em `views/executivo/inicio.tsx`	S2, S3	6 a 8 h
S5	Painel financeiro editorial: contexto financeiro completo e região na visão geral, com séries, categorias, contas por cliente e alertas derivados	S4	8 a 12 h
S6	Central de decisões contextual: prévia, lista com filtros, gaveta com foco preso e votação real via castVote; pedir contexto vira abstenção com comentário	S4	8 a 12 h
S7	Ecossistema e núcleo orbital: `orbital-engine.ts` portado, componente de canvas, derivador de nós e arestas reais, presets de câmera por contexto	S2, S3	10 a 14 h
S8	Modos e camada cinematográfica: provedor de modo com persistência, grão, vinheta e letterbox ativos apenas no Imersivo, alternador no cabeçalho	S4	5 a 7 h
S9	Cap no novo palco e paleta estendida: reskin do palco da Cap na tela do CEO (componente da Cap intocado) e comandos novos na paleta de comandos	S4	4 a 6 h

Etapa	Entregas e arquivos afetados	Depende de	Esforço
S10	Testes novos: derivações, estados honestos por região, provedor de modo, orbital estático em movimento reduzido, navegação de contextos	S1 a S9	6 a 9 h
S11	Gates finais e evidências: execução integral da qualidade obrigatória, correções finais, relatório com resultados medidos e flag liberada para avaliação	S10	5 a 8 h

4.4 Gates de qualidade obrigatórios — como serão executados e apresentados

Cada gate tem comando, critério objetivo e evidência entregue. Nada é autoavaliado: o que o PROMPT 05 pede para executar e apresentar será apresentado como saída de ferramenta — log, captura ou relatório — e o relatório final separa explicitamente o que foi medido do que permanece pendente de validação por falta de ambiente. A última execução integral conhecida da base é a da Etapa 1.1, com zero erros de lint e de tipos, 84 testes aprovados, build com validação de tipos e verificação em navegador; a Fase 3 repete esse protocolo sobre a base estendida.

Tabela 13 — Gates obrigatórios — método, critério e evidência

Verificação	Método e comando	Critério de aprovação	Evidência apresentada
ESLint	`bun run lint`	Zero erros	Log integral da execução
TypeScript	`bun run typecheck` (tsc sem emissão)	Zero erros	Log integral da execução
Build	`bun run build`	Aprovado com validação de tipos ativa	Log de build com a etapa de tipos presente
Testes automatizados	`bun run test`	Suítes existentes preservadas e novas aprovadas; sete de RLS aprovados quando as variáveis existirem na data	Log com contagens por suíte
Responsividade	Protocolo nos formatos 390, 768 e 1440 pixels, reutilizando os roteiros de QA do PROMPT 04	Zero overflow horizontal; alvos de toque mantidos	Capturas de tela com as medidas registradas

Verificação	Método e comando	Critério de aprovação	Evidência apresentada
Acessibilidade	Contraste medido por script (reuso do medidor dos QA do protótipo); caminhada de teclado; movimento reduzido	Contraste AA nos pares obrigatórios; foco visível; Esc por camada; orbital estático em movimento reduzido	Relatório de contraste com valores por par e roteiro de teclado registrado
Desempenho	Lighthouse mobile sobre build de produção; delta de bundle; quadros por segundo do orbital com medidor interno	Performance igual ou acima de 85; acessibilidade igual ou acima de 90; delta de bundle documentado	Relatório Lighthouse em JSON e medições anotadas

Endurecimento futuro, fora do escopo e recomendado: as aproximadamente trinta regras de ESLint desligadas no config atual e o any implícito permitido no tsconfig merecem revisão dedicada após a prova de conceito — registrados aqui para não se perderem.

4.5 Aceite e rollback da Fase 3

O aceite é uma lista curta e verificável, sem espaço para interpretação:

1. Todos os gates verdes, com evidências de ferramenta anexadas ao relatório final.
2. Matriz de preservação verificada item a item, com fumaças registradas.
3. Estados reais confirmados em cada região da nova tela, incluindo não configurado.
4. Flag ligada apenas para avaliação do contratante, com composição atual ainda presente no código.
5. Relatório final separando o medido do pendente de validação, com motivos de cada pendência.

O rollback segue os critérios objetivos da seção 2.7: qualquer gatilho dispara a reversão da etapa correspondente, com a flag desligada por padrão mantendo os usuários na composição atual o tempo todo. Se o contratante aprovar, a extensão da linguagem às demais telas passa a ser escopo de nova solicitação — a decisão de rollout não pertence a esta fase.

5. Consolidado — Esforço, Sequência e Marcos

5.1 Matriz de esforço consolidada

As faixas abaixo cobrem engenharia de execução — planejamento, código, testes e gates — e excluem o tempo de avaliação do contratante, o fornecimento de insumos externos e qualquer rollout além da tela inicial do CEO. Premissas: um único desenvolvedor sênior familiarizado com a base, ambiente com as ferramentas já instaladas, revisões feitas na sequência sem fila, precisão declarada de mais ou menos trinta por cento.

Tabela 14 — Esforço por item, com faixas e notas

Item	Faixa	Notas de estimativa
Fase 1 — planejamento de integração	Entregue	É o conteúdo dos capítulos 2 a 4 deste documento; zero hora de engenharia restante.
Fase 2 — runbook de validação RLS	2 h	Roteiro passo a passo materializado em docs/VALIDACAO-RLS.md.
Fase 2 — auditoria de segredos e varredura	1 a 2 h	Roteiro executável de varredura estática e conferência de coberturas.
Fase 2 — revisão formal das migrations	1 h	Revisão já executada analiticamente aqui; o restante é registro formal junto às evidências.
Fase 2 — execução da suíte RLS e evidências	2 a 3 h	Com banco de teste fornecido; inclui diagnóstico se qualquer teste falhar.
Subtotal Fase 2	6 a 8 h	Bloqueada até autorização e insumos do contratante.
Fase 3 — S1 a S11 (capítulos 4.3)	66 a 97 h	Onze etapas com PR atômico cada; faixa reflete a incerteza normal de composição visual.
Fase 3 — reserva de contingência	10 a 15 h	Quinze por cento sobre o teto da faixa, para correções de gate.
Subtotal Fase 3	76 a 112 h	Aproximadamente onze a dezesseis dias úteis de um desenvolvedor sênior.
Total geral de engenharia	82 a 120 h	Somatório das fases autorizáveis; não inclui rollout às demais telas.

5.2 Sequência com pontos de autorização

O plano avança por marcos, e cada marco tem um gatilho claro — quem decide e o que desbloqueia. Nenhuma execução começa sem o gatilho correspondente, cumprindo a exigência do PROMPT 05 de aguardar autorização, com a Fase 3 exigindo autorização específica e adicional à da Fase 2.

Tabela 15 — Marcos, gatilhos e entregáveis

Marco	Gatilho	Entregável
M0	Entrega do PROMPT 05 (agora)	Este plano técnico — nenhum arquivo do aplicativo alterado.

Marco	Gatilho	Entregável
M1	Autorização expressa da Fase 2 e insumos: projeto de teste com quatro usuários e variáveis	Runbook de validação, auditoria de segredos executada, suíte RLS rodada com log integral e resultado sete aprovados.
M2	Autorização específica da Fase 3	Execução S1 a S11 com PR atômico por etapa e flag desligada.
M3	Gates verdes da Fase 3	Relatório final com evidências medidas, flag liberada para avaliação e matriz de preservação verificada.
M4	Decisão do contratante	Aprovação: extensão da linguagem às demais telas vira nova solicitação; ou rollback conforme seção 2.7.

Dependências entre marcos: M1 exige os insumos do contratante e nada mais; M2 é recomendada somente após M1 verde, porque a suíte RLS é a rede de segurança da fronteira de dados usada pela nova tela; M3 exige ambiente de publicação para os gates de Lighthouse — sem ele, a medição é feita em build local com a ressalva registrada no relatório. O caminho crítico do início de M1 ao relatório de M3 é a soma das faixas: cerca de onze a dezesseis dias úteis de engenharia, intercalados com as revisões do contratante.

5.3 PENDENTE DE VALIDAÇÃO — verificado agora x medível depois

Nos termos da disciplina do projeto — resultados medidos, nunca presumidos —, a tabela abaixo separa o que foi verificado de fato nesta entrega daquilo que só poderá ser medido após as autorizações e os insumos. Nada da lista abaixo é tratado como concluído; o relatório da Fase 3 atualizará esta lista item a item com os valores medidos.

Tabela 16 — Situação atual dos itens de validação

Item	Situação verificada agora	Quando poderá ser medido
Sete testes de integração RLS	Suíte pronta; pula sozinha sem variáveis; sete pulados na última execução documentada	Fase 2, com o banco de teste do contratante
CRUD real contra Supabase	Camada pronta e testada para recusar com clareza sem configuração	Assim que .env.local existir
Lighthouse da nova tela	Inexistente por projeto: o aplicativo ainda não tem a tela nova	Fase 3, etapa S11
Contraste medido da nova interface	Escada do protótipo medida nos QA do PROMPT 04; no app, nada a medir ainda	Fase 3, etapa S11
Overflow em 390 pixels da nova composição	Protótipo mobile verificado em 390; composição no app ainda não existe	Fase 3, etapa S11

Item	Situação verificada agora	Quando poderá ser medido
Atualização do PWA após troca de shell	Estratégia de cache correta e verificada na fundação	No release da Fase 3, com versão de cache elevada
Quadros por segundo do orbital no aplicativo	Motor medido nos QA do protótipo com degradação adaptativa	Fase 3, em dispositivo real
Estado do repositório, migrations e testes	Verificado nesta entrega, por leitura direta: fundação íntegra e endurecida	Confirmado em cada marco

5.4 Próximos passos

O plano está pronto para decisão. A sequência recomendada de ações, na ordem exata em que cada uma se desbloqueia:

1. O contratante revisa este plano e ajusta premissas que julgar necessárias — especialmente a decisão do risco R13 sobre o protótipo na pasta pública.
2. Autorização expressa da Fase 2, acompanhada dos insumos: projeto Supabase de teste descartável com as migrations e os quatro usuários, além das variáveis de ambiente.
3. Execução da Fase 2 com evidências: runbook, auditoria de segredos e a suíte RLS completa com sete testes aprovados.
4. Autorização específica da Fase 3, que inicia a sequência S1 a S11 com PR atômico por etapa e flag desligada até o fim.
5. Relatório final com todos os gates executados e evidências anexadas, separando o medido do pendente — e a avaliação do contratante com a flag ligada.

Até a próxima autorização, nada será executado no aplicativo: nenhuma alteração em código, nenhuma migration rodada, nenhuma conta criada, nenhum dado gerado. Este documento é a única entrega da solicitação e permanece como referência vinculante para as fases que ele descreve.